INFORME DE LABORATORIO

LABORATORIO 03

Funciones, módulos y programación orientada a objetos

Aprendiz	Yohan Fernando Cuellar Ospino
Ficha	3388087
Repositorio	https://github.com/FIRE31690/Laboratorios_Python
Fecha del informe	23 de octubre de 2026

Propósito

Documentar las prácticas del tercer laboratorio de Python, organizadas alrededor de funciones, módulos, clases, objetos, encapsulamiento y colaboración entre objetos. El paquete incluye un gestor de inventario como reto integrador.

1. Actividades desarrolladas

Práctica 3.1 — Funciones

- Definición de funciones, parámetros y argumentos.
- Diferencia entre mostrar resultados con **print()** y devolver valores con **return**.
- Alcance de variables, valores predeterminados y trabajo con listas.
- Separación de cálculos y lógica para facilitar la reutilización.

Práctica 3.2 — Módulos e importaciones

- Separación del programa en varios archivos y uso de importaciones.
- Diferencias entre **import modulo** y **from modulo import funcion**.
- Uso de **if __name__ == "__main__"** para controlar el punto de entrada.
- Validaciones y organización de un cotizador modular.

Práctica 3.3 — Clases y objetos

- Creación de clases, instancias, atributos y métodos.
- Uso de **__init__** para inicializar objetos y de **self** para acceder a sus atributos.
- Prevención de listas compartidas accidentalmente entre instancias.

Práctica 3.4 — Encapsulamiento y colaboración

- Uso de propiedades y métodos para proteger las reglas de los objetos.
- Separación de las responsabilidades de Producto e Inventario.
- Validación de existencias y coordinación de objetos relacionados.

2. Reto integrador: gestor de inventario orientado a objetos

La aplicación transforma el gestor basado en diccionarios del laboratorio anterior en una estructura con clases. El menú y la interacción se concentran en main.py; producto.py define los productos e inventario.py administra la colección.

Responsabilidades principales

- **Producto:** guarda código, nombre, categoría, precio y cantidad; valida datos y calcula el valor de sus existencias.
- **Inventario:** registra productos sin duplicar códigos, lista, busca, actualiza, elimina, filtra productos sin stock y calcula el resumen.
- **main.py:** muestra el menú, recibe entradas, repite solicitudes cuando hay datos numéricos inválidos y comunica los resultados.

3. Pruebas y comprobaciones

Se ejecutaron localmente los dos archivos de pruebas incluidos en gestor_inventario.

Prueba	Resultado observado
pruebas_inventario.py	Terminó correctamente y mostró “Todas las pruebas pasaron correctamente”.
pruebas_independencia.py	Terminó correctamente; mostró “Primero: 1” y “Segundo: 0”, confirmando que los inventarios mantienen sus propias listas.

Casos incluidos en la documentación del proyecto

- Rechazo de códigos duplicados, nombres vacíos, categorías no permitidas, precios no positivos y cantidades negativas.
- Búsqueda de productos existentes e inexistentes, actualización y eliminación.
- Conservación del valor anterior cuando se intenta actualizar una cantidad inválida.
- Resumen de inventario: 3 productos, 6 unidades, valor total $1.740.000, 2 categorías y producto Mouse sin stock.

4. Depuración

La documentación registra errores provocados y sus correcciones: listas mutables compartidas a nivel de clase, atributos inexistentes, métodos sin self, falta de validación en el constructor, importaciones ausentes y cálculo del inventario sin multiplicar precio por cantidad.

5. Conclusión

El laboratorio organiza el código en funciones y módulos y después en clases con responsabilidades definidas. El reto integrador aplica encapsulamiento y validaciones para mantener los datos del inventario consistentes. Las dos pruebas ejecutadas terminaron correctamente en el entorno de preparación.

Nota de entrega: el resultado de las pruebas se verificó localmente. La publicación en GitHub y el historial real de commits deben comprobarse por separado antes de entregar.